

MARCO UNIFICADO DE EVALUACIÓN TÉCNICA, METODOLÓGICA Y MENTORÍA (CON OBSERVACIONES)

Caso de Estudio: IKernell Soluciones Software (Modernización ADSI)
Stack de Evaluación: Java Spring Boot, React, PostgreSQL, Git & GitHub

Estimado candidato, el documento que tienes en tus manos es el resultado final y consolidado de un proceso riguroso de auditoría y acompañamiento. Es muy importante que comprendas que lo que verás plasmado aquí es solo la fase de cierre: el trabajo fuerte de evaluación ya fue ejecutado previamente por el equipo de pares evaluadores.

Para llegar a las conclusiones, observaciones y planes de mejora que encuentras en las siguientes secciones, los evaluadores realizaron un proceso exhaustivo en tres frentes:

Auditoría Técnica y de Requisitos: Se analizaron a fondo tus Historias de Usuario, la lógica de tus diagramas UML y la normalización de tu base de datos en PostgreSQL.

Revisión de Código (Code Review): El equipo de ingeniería inspeccionó minuciosamente tu repositorio de GitHub, evaluando la arquitectura limpia en Java Spring Boot, el manejo de la seguridad, la construcción de componentes en React, tus hojas de estilo CSS y la semántica de tus commits.

Sesiones de Entrevista: Se contrastaron tus decisiones de diseño y desarrollo con los lineamientos de la industria actual.

Puesto de la siguiente manera:

SECCIÓN 1: GUÍA METODOLÓGICA PARA EVALUAR

El proceso de evaluación consta de 4 fases secuenciales orientadas a validar tanto las competencias técnicas duras (*hard skills*) como el potencial de crecimiento y actitud del candidato (*soft skills*).

- FASE 1: Auditoría Documental —► Revisión de HUs, UML, Diagramas y Modelo Relacional.
- |
- FASE 2: Análisis de Código —► Revisión de Backend, Frontend, PostgreSQL y Repo Git.
- |
- FASE 3: Entrevista Técnica —► Validación de decisiones de diseño y cuestionario humano.
- |
- FASE 4: Feedback de Mentoría —► Entrega de reporte constructivo y plan de crecimiento.

SECCIÓN 2: Rúbricas Unificadas y Matrices de Evaluación

Esta sección representa el núcleo técnico de la revisión. Aquí es donde el equipo evaluador analiza a fondo el proyecto que construiste. Se divide en matrices que evalúan las cinco áreas clave de la ingeniería de software:

1. **Ingeniería de Requisitos y Diseño:** Cómo estructuraste tus Historias de Usuario y diagramas.
2. **Base de Datos (PostgreSQL):** La lógica, normalización y buenas prácticas en tus scripts de base de datos.
3. **Backend (Java Spring Boot):** La arquitectura de tu código, la seguridad y el manejo de datos.
4. **Frontend y Maquetación (React y CSS):** La usabilidad de tu interfaz, rutas protegidas y adaptabilidad visual.
5. **Repositorio (Git y GitHub):** El orden de tus carpetas y las buenas prácticas en tu historial de cambios.

💡 **Nota para el candidato:** En cada matriz, los evaluadores no solo marcarán si el requerimiento cumple o no, sino que te dejarán **Observaciones Técnicas** detalladas con los lineamientos exactos de la industria para que sepas cómo llevar tu código al siguiente nivel.

SECCIÓN 5: Reporte de Feedback Técnico y Profesional

Esta es la sección de cierre y la más valiosa para tu carrera. Una vez concluidas las fases de revisión, los evaluadores consolidarán un informe personalizado para ti. En él encontrarás:

- Tus **fortalezas** principales identificadas durante el reto.
- Tus **áreas de crecimiento** (puntos específicos que debes reforzar).
- Un **Plan de Acción** con temas sugeridos y herramientas reales de la industria para que continúes estudiando.
- Los pasos formales a seguir para la entrega de tus correcciones y la continuidad en la Fábrica de Software.

SECCIÓN 2: RÚBRICAS UNIFICADAS Y MATRICES DE EVALUACIÓN

MATRIZ DETALLADA A.1: Auditoría de Historias de Usuario (HUs)

SUB-ÍTEM	COMPONENTE EVALUADO	CUMPLE (SÍ/NO/ PARCIAL)	OBSERVACIONES / EVIDENCIA / MEJORA REQUERIDA	TÉCNICAS Y METODOLÓGICAS
A.1.1	Estructura de Cabecera y Metadata	SÍ		Código: Formato HU- [Número] secuencial. Nombre: Título descriptivo corto. Complejidad: Clasificación técnica (Alta/Media/Baja). HU Relacionada: Vínculo a dependencias entre historias.
A.1.2	Descripción de Valor de Negocio	SÍ	HU 01 Y HU 02: Definir mismo nombre para rol interesado / anónimo	El bloque de descripción debe cumplir estrictamente con: Yo como: Rol del sistema Requiero: La necesidad funcional sin tecnicismos. Para: El beneficio directo del negocio. Requerimiento

A.1.3	Criterios de Aceptación (Gherkin Positivos)	PARCIAL	HU 01 CONDICIÓN 03: En vez de enviar directamente al home page, que exista una vista de ‘no tiene acceso’ o “no existe “ y en esas vistas que exista un botón para redirigir ahí sí al homepage. HU 02 Condición 01: Actualizar nombre de botones y campos que corresponda con la interfaz. HU 02: Corregir, no funciona la lógica de preguntás como rol interesado. Escenarios positivos faltan validaciones 05-C01...: validacion faltante campo edad minima para trabajar	Happy Path (Condiciones de éxito): Escenarios positivos estructurados
A.1.4	Criterios de Aceptación (Gherkin Negativos)	PARCIAL	Añadir y estructurar condiciones negativas dentro de cada una de las historias de usuario, siguiendo el formato de la plantilla SENA para Historias de Usuario Pensarlo como:	Edge Cases (Condiciones de error/fallo): Escenarios negativos estructurados en <i>Dado/Cuando/Entonces</i> que detallen cómo reacciona el sistema ante fallas (ej: registrar datos duplicados, vacíos o inválidos).- ’ -

			Escenario: Intento de inicio de sesión con contraseña incorrecta Dado que: El usuario se encuentra en la pantalla de login con una cuenta registrada. Cuando: Introduce su correo electrónico correcto pero escribe una contraseña incorrecta, y hace clic en "Iniciar Sesión". Entonces: El sistema deniega el acceso, no genera sesión, y muestra el mensaje de error: "El correo electrónico o la contraseña son incorrectos".	
A.1.5	Granularidad y Especificidad de Tareas	PARCIAL	Aumentar el nivel de especificidad y granularidad. Más detalle en la tarea por item pero a la vez pensar las tareas como los pasos que necesito completar para que mi historia de usuario se cumpla. ESTO EN TODAS LAS HISTORIAS.	Rechazar tareas genéricas como "hacer backend" o "crear diseño". Debe auditarse que las tareas estén desglosadas técnicamente: • <i>Ej. Frontend:</i> "Diseñar formulario React con validación de expresiones regulares para correo, maquetar

			Pensarlo como: cuales son cada una de esas tareas que debo cumplir para que mi historia de usuario de dé por terminada.	con CSS Flexbox y crear handler para submit". • <i>Ej. Backend:</i> "Crear endpoint POST <code>/api/usuario</code>, escribir servicio JPA, mapear DTO y añadir anotaciones <code>@Valid</code>".
--	--	--	--	---

MATRIZ A: Ingeniería de Requisitos y Diseño (UML e Historias de Usuario)

SUB-ÍTEM	CRITERIO DETALLADO DE CUMPLIMIENTO	CUMPLE (SÍ/NO/ PARCIAL)	EVIDENCIA / MEJORA REQUERIDA	OBSERVACIONES TÉCNICAS
A.2	Casos de Uso Extendidos: El diagrama identifica actores y modela con precisión relaciones <<include>> (ej. Autenticar) y <<extend>>	SÍ	NO	NO
A.3	Diagrama de Clases: Estructura de herencia de tabla base para las personas en el software. Representación de multiplicidades 1:N y N:M.	SÍ	Maneja multiplicidad y también tiene tipos de relación, ejemplo: composición--	

MATRIZ B: Base de Datos (PostgreSQL)

SUB-ÍTEM	CRITERIO DETALLADO DE CUMPLIMIENTO	CUMPLE (SÍ/NO/ PARCIAL)	EVIDENCIA / MEJORA REQUERIDA	OBSERVACIONES TÉCNICAS
B.1	Normalización (3FN): El diseño evita la redundancia. Tablas como Errores, Interrupciones e HistorialTrabaj o están correctamente desacopladas de las tablas core (Trabajador, Proyecto).	PARCIAL	Unificó la personas en la tabla usuario pero FALTA la tabla intermedia entre rol y usuario para permitir más de un rol por usuario	NO
B.2	Script DDL Limpio: Sentencias estructuradas en orden lógico de ejecución, definición explícita de llaves primarias (PK), foráneas (FK) y tipos de datos correctos en PostgreSQL.	PARCIAL	Revisar la nomenclatura para nombrar es base de datos. partes de campos está Ok.	Nomenclatura en Base de Datos: Reglas Clave • Minúsculas y Snake Case: Todo el esquema (tablas y columnas) va en minúsculas, separando palabras con guión bajo (ej. detalle_factura). • Sistemas Operativos: Evita errores de compatibilidad en entornos como Linux, que sí es sensible a

				mayúsculas y minúsculas. ● Plurales y Singulares: Nombra las tablas en plural (usuarios) y sus campos o columnas en singular (correo_electronico). ● Limpieza: Prohibido usar espacios, eñes (ñ), tildes o caracteres especiales para prevenir fallos en las consultas SQL.
B.3	Integridad y Rendimiento: Definición de restricciones de integridad (NOT NULL, UNIQUE, CHECK). Creación de FKs y campos con alta frecuencia de consulta.	SÍ	NO	NO
B.4	Transaccionalidad: Uso adecuado de transacciones para operaciones complejas (ej. asignaciones de líderes como desarrolladores en proyectos simultáneos).		[Especificar scripts o métodos JPA que requieran transacción]	Es indispensable asegurar que las transacciones sean atómicas para evitar inconsistencias de datos de la base de datos.

MATRIZ C: Backend (Java Spring Boot)

SUB-ÍTEM	CRITERIO DETALLADO DE CUMPLIMIENTO	CUMPLE (SÍ/NO/ PARCIAL)	EVIDENCIA / MEJORA REQUERIDA	OBSERVACIONES TÉCNICAS
C.1	Manejo Dinámico de Roles (Security): Spring Security configurado con JWT. Endpoints restringidos usando anotaciones como <code>@PreAuthorize("hasRole('COORDINADOR')")</code> .	PARCIAL	Poner las rutas de los endpoint que puede ver cada usuario. Para cada rol identificar que tipo de petición se puede hacer .	Depende de las reglas de negocio establecidas por el desarrollador.
C.2	Arquitectura Controller-Service-Repository: Los controladores no contienen lógica de negocio ni llamadas directas a SQL/JPQL. La lógica reside únicamente en componentes <code>@Service</code> .	Sí	[Especificar clases donde	

C.3	Manejo Global de Excepciones: Clase anotada con <code>@RestControllerAdvice</code> que captura excepciones comunes y retorna JSONs con mensajes y códigos HTTP adecuados.	Sí	El arquitecto a n capas Secrear una conf. .. .	Se prohíbe exponer trazas internas de error SQL o de compilación directamente en el JSON de respuesta del cliente.
C.4	Validación de Datos de Entrada: Uso de anotaciones de validación (ej. <code>@NotNull</code> , <code>@Size</code> , <code>@Email</code> , <code>@Past</code>) en los DTOs de entrada de los controladores.	NO	Especificar DTOs con validaciones faltantes	La anotación <code>@Valid</code> debe estar presente en los parámetros de los métodos del <code>@RestController</code> .
C.5	Auditoría Automática de Entidades (Opcional): Implementación de auditoría JPA (ej: <code>@CreatedDate</code> , <code>@LastModifiedBy</code>) en tablas críticas (ej. proyectos, asignaciones).	NO	[Especificar entidades que carezcan de metadatos de auditoría]	Clave para realizar el seguimiento del historial de cambios de estados de proyectos y perfiles por parte de los coordinadores.

MATRIZ D: Frontend, Usabilidad, CSS y Maquetación Semántica

SUB-ÍTEM	CRITERIO DETALLADO DE CUMPLIMIENTO	CUMPLE (SÍ/NO/ PARCIAL)	EVIDENCIA / MEJORA REQUERIDA	OBSERVACIONES TÉCNICAS
D.1	Rutas Protegidas: Implementación de componentes de enrutamiento protegido (ej. <code><ProtectedRoute ></code>) que impiden el renderizado de interfaces a usuarios no autorizados.	PARCIAL	va de la mano con lo que cada usuario puede ver según su rol asignado.	Validar que la redirección del navegador funcione de manera inmediata al limpiar la sesión del cliente.
D.2	Usabilidad y Flujos de Error: Validación visual de formularios en tiempo real o luego de la acción. Feedback claro ante operaciones exitosas o fallidas (mensajes de carga,	PARCIAL	Especificar mensajes de feedback ejemplo en login si no ingreso campos y hago clic en evento ingresar.	Debe validarse que el usuario final no quede bloqueado ante un fallo del servidor (la interfaz debe informar la novedad).
D.3	Estilos CSS3, Flexbox/Grid y Responsividad	PARCIAL	Estandarizar estilos en componentes. Evaluar el layout: Uso correcto de propiedades de alineación en Flexbox y Grid CSS. Estructura responsiva (Media Queries o CSS dinámico)	Componentes Estándar y Limpios: Diseña clases modulares y reutilizables, agrupando las propiedades de forma lógica para evitar código duplicado y facilitar el mantenimiento.

			adaptándose a tablets y móviles. Código CSS ordenado, limpio y sin duplicaciones de propiedades innecesarias.	● Layout Eficiente (Flexbox y Grid): Usa Flexbox para estructuras unidimensionales (alineación y distribución en filas/columnas) y Grid para layouts bidimensionales complejos (filas y columnas simultáneas), aplicando siempre propiedades nativas de alineación (justify-content, align-items). ● Responsividad Dinámica: Diseña bajo el enfoque Mobile-First, implementando Media Queries estratégicas o CSS dinámico (clamp(), calc(), minmax()) para lograr una adaptabilidad fluida en tablets y móviles.
--	--	--	---	--

MATRIZ E: Repositorio y Versionamiento en GitHub (Estándar Profesional)

SUB-ÍTEM	CRITERIO DETALLADO DE CUMPLIMIENTO	CUMPLE (SÍ/NO/ PARCIAL)	EVIDENCIA / MEJORA REQUERIDA	OBSERVACIONES TÉCNICAS
E.1	Estructura Organizacional del Repositorio: El repositorio presenta una estructura de carpetas limpia: <code>/backend,</code> <code>/frontend,</code> <code>/database (scripts SQL), y /docs.</code>	SÍ	NO	NO
E.2	Historial y Mensajes de Commit Semánticos: Mensajes bajo el estándar <i>Conventional Commits</i> (ej: <code>feat(auth):</code> <code>implementar JWT,</code> <code>fix(db):</code> <code>corregir</code> <code>constraint).</code>	PARCIAL	Estandarizar buenas prácticas de títulos commit. Especificar ejemplos de commits con nombres genéricos añadiendo convenciones como feat, fix u otros. Evitar mensajes ambiguos como "actualización", "corrigiendo errores" o "cambios finales".	Teoría y Estándar de Commits Semánticos • Estructura fija: Los mensajes deben usar el estándar. • Conventional Commits con la estructura tipo(contexto): descripción corta en minúsculas, asegurando un historial limpio y automatizable. • Tipos clave (Ejemplos): Usa

				prefijos claros como feat(auth): implementar JWT para nuevas características, fix(db): corregir constraint para solución de errores, o docs(readme): actualizar guía para documentación. ● Prohibido el contenido ambiguo: Se descartan por completo mensajes genéricos como "actualización", "corrigiendo errores" o "cambios finales" debido a que destruyen la trazabilidad del código.
E.3	Archivo .gitignore Configurado: Exclusión explícita de archivos de compilación, IDEs, dependencias (/node_modules, /target) y variables de entorno (.env).	Sí	[Especificar archivos basura o credenciales subidas]	Es una falta crítica de seguridad el subir credenciales de base de datos o claves secretas de JWT al repositorio público.

E.4	Documentación del Repositorio (README.md Opcional): Archivo descriptivo con instrucciones detalladas de instalación local, prerequisites, variables de entorno y comandos de ejecución.	SÍ	NO	NO
-----	--	----	----	----

SECCIÓN 5:

REPORTE DE FEEDBACK TÉCNICO Y PROFESIONAL

Candidato: EFRAÍN MANOTAS

Evaluador 1: BRANNDON SUÁREZ

Evaluador 2: SONIA CANO

Fecha de Evaluación: 01 JULIO 2026 - 03 JUL 2026

Resultado de Selección: [Aprobado con observaciones e items por mejorar]

1. Fortalezas Identificadas

- **Documentación, Análisis y Modelado:** Se evidencia una adecuada capacidad para documentar requerimientos, modelar soluciones técnicas y adaptar la documentación a los formatos solicitados, manteniendo claridad y organización.
 - **Desarrollo y Arquitectura de Software:** Se observa una correcta aplicación de conceptos de programación, organización del proyecto y separación de responsabilidades mediante arquitecturas por capas, demostrando capacidad para adoptar nuevas tecnologías como Spring Boot.
 - **Base de Datos Relacionales:** Se evidencia comprensión de los fundamentos del modelado relacional, definición de entidades, relaciones y estructura de bases de datos.
 - **Control de Versiones:** Uso adecuado de Git para el seguimiento de cambios y organización del desarrollo del proyecto.
-

2. Áreas de Crecimiento y Oportunidades de Mejora

- **Documentación, Análisis y Diseño Funcional:** Fortalecer la definición de Historias de Usuario mediante criterios de aceptación completos utilizando el enfoque BDD (Given-When-Then), documentando tanto los escenarios exitosos (Happy Path) como los escenarios alternativos y de excepción (Unhappy Path). Se recomienda complementar el análisis funcional mediante herramientas de modelado como Sketches, Wireframes o Mockups para validar previamente la experiencia de usuario.
- **Base de Datos Relacionales:** Reforzar las buenas prácticas de diseño y nomenclatura adoptando convenciones consistentes, como nombres en minúsculas, formato snake_case, tablas en plural y columnas en singular, favoreciendo la mantenibilidad, legibilidad y compatibilidad entre entornos.
- **Backend y Seguridad:** Implementar un manejo centralizado de excepciones evitando exponer información sensible al cliente, complementándolo con validaciones de entrada, uso de DTOs y respuestas HTTP consistentes para fortalecer la seguridad y robustez de la aplicación.

- **Frontend y Arquitectura CSS:** Optimizar la organización de estilos mediante componentes reutilizables, reducir la duplicidad de código y fortalecer la implementación de interfaces responsivas utilizando adecuadamente CSS Grid y Flexbox, manteniendo coherencia con los requerimientos funcionales y la experiencia de usuario.
 - **Control de Versiones:** Adoptar estándares como Conventional Commits para la creación de mensajes de commit descriptivos, mejorando la trazabilidad de los cambios, la colaboración entre equipos y la integración continua.
-

3. Plan de Acción y Recursos Sugeridos para tu Carrera

Queremos que este proceso sea un escalón para tu crecimiento. Te sugerimos enfocarte en los siguientes temas para tus siguientes retos de desarrollo:

- **Documentación, Análisis y Diseño Funcional**

Temas sugeridos:

- Historias de Usuario y criterios de aceptación mediante BDD.
- Modelado de flujos funcionales.
- Wireframes, Mockups y prototipado temprano.

Recursos recomendados

- Jira.
- Azure DevOps.
- Cucumber.
- Gherkin.
- Figma.
- Stitch AI

- **Base de Datos Relacionales**

Temas sugeridos:

- Convenciones de nomenclatura.
- Normalización.
- Diseño de modelos relacionales.

Recursos recomendados

- PostgreSQL Documentation.
- SQLFluff.
- DBeaver.

- pgAdmin 4.

- **Backend y Seguridad**

Temas sugeridos

- Manejo global de excepciones.
- Validación de DTOs.
- Buenas prácticas de seguridad para APIs REST.

Recursos recomendados

- Documentación oficial de Spring Boot (Validation y Error Handling).

- **Frontend y Arquitectura CSS**

Temas sugeridos

- CSS Grid.
- Flexbox.
- Diseño responsivo.
- Componentización y reutilización de estilos.

Recursos recomendados

- MDN Web Docs.
- CSS-Tricks.
- Documentación oficial de Bootstrap.

- **Control de Versiones**

Temas sugeridos

- Conventional Commits.
- Git Flow.
- Estrategias de trabajo colaborativo.

Recursos recomendados

- ConventionalCommits.org.
 - Documentación oficial de Git.
 - Commitlint.
 - Husky.
-

4. Mensaje de Cierre y Agradecimiento

Apreciado aprendiz Efraín Manotas:

Queremos agradecer sinceramente el tiempo, la dedicación y el compromiso que invertiste en el desarrollo de este reto técnico. Valoramos especialmente la disposición que demostraste para abordar un ejercicio de carácter técnico, así como el esfuerzo por construir una solución funcional y estructural.

Durante el proceso también destacamos tu apertura para recibir retroalimentación, tu capacidad para identificar oportunidades de mejora y la disposición para realizar ajustes de forma iterativa, cualidades que reflejan una actitud de aprendizaje continuo y crecimiento profesional.

Esperamos que el feedback compartido en este documento te sirva como una guía para fortalecer tus conocimientos y continuar desarrollando habilidades que serán de gran valor en futuros proyectos y procesos de selección. Así como los espacios de revisión, evaluación, preguntas e inquietudes durante la revisión del reto.

En los próximos días esperamos recibir las correcciones y ajustes derivados de las observaciones consignadas en este informe de parte tuya. Esta entrega nos permitirá evidenciar la aplicación del feedback recibido y complementar la evaluación integral del reto técnico.

Una vez revisadas las correcciones y finalizada esta etapa de evaluación, el instructor a cargo de la Fábrica de Software, Diego Boada, dará continuidad a las siguientes fases del proceso y se comunicará contigo para informar los pasos correspondientes.

Agradecemos nuevamente tu participación, el profesionalismo con el que asumiste este reto y la actitud demostrada durante todo el proceso. Independientemente del resultado final, confiamos en que este ejercicio represente una oportunidad para seguir fortaleciendo tus competencias técnicas y profesionales.

Te deseamos muchos éxitos en tu crecimiento como desarrollador(a) de software y esperamos poder coincidir nuevamente en futuros proyectos o procesos de selección.